

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Отчёт по курсовой работе

по курсу «Архитектура программных систем»

Выполнил

Линь Цзэдун

Преподаватель

Гончаров А.В.

Санкт-Петербург
2026

Содержание

Введение.....	3
Определение требований.....	3
Функциональные требования.....	3
Нефункциональные требования.....	4
Расчёт ресурсов	6
Бизнес- и прикладные домены	7
Выбор технологий и проектирование	8
High-level design	8
Low-level design	8
Описание используемых технологий	9
Разработка распределённой системы	10
Описание схемы данных.....	10
Описание контрактов серверного API.....	12
Интерфейс взаимодействия с конечным пользователем	13
Тестирование	17
Архитектурное соответствие нефункциональным требованиям	17
Нагрузочные испытания	18
Стресс-/capacity-тестирование	19
Нагрузка x10 от плановой.....	21
Эксплуатация	22
Расчёт доступности	22
Анализ узких мест	23
Выводы	24

Введение

URL Shortener - распределённая автоматизированная система для преобразования длинных URL в короткие ссылки. Сокращённые ссылки упрощают распространение адресов в мессенджерах, социальных сетях, учебных материалах и рабочих документах. Для такого сервиса критичны быстрый редирект, устойчивое хранение соответствия `route_key` → исходный URL, контроль уникальности `alias` и безопасная обработка ошибок.

Выбранный бизнес-домен является *read-heavy*: переходов по коротким ссылкам существенно больше, чем операций создания и управления ссылками. Поэтому архитектура строится вокруг изоляции быстрого `redirect-path`, кэширования горячих `route_key` и асинхронного сбора аналитики.

Состав ролей команды: аналитик требований, архитектор распределённой системы, разработчик прототипа, инженер нагрузочного тестирования и DevOps-инженер. Собственная роль в рамках работы: архитектор и разработчик прототипа с участием в расчётах ресурсов и нагрузочных испытаниях.

Базовая цель работы: получение опыта работы в кросс-функциональной команде в рамках полного цикла создания распределённой автоматизированной системы. Дополнительная цель: спроектировать и реализовать учебный MVP URL Shortener, проверить его архитектурные решения расчётом ресурсов, схемой данных, расчётом доступности и нагрузочными тестами.

Определение требований

В реализацию включены все требования, зафиксированные в таблице требований. Ниже приведены бизнес-, функциональные и нефункциональные требования, по которым строились архитектура, прототип и нагрузочные проверки.

Функциональные требования

Таблица 1. Бизнес-требования

№	Бизнес-требование
1	Предоставить пользователям возможность преобразовывать длинные URL в короткие ссылки для удобного распространения в мессенджерах, социальных сетях и учебных/рабочих материалах.
2	Обеспечить быстрый и корректный редирект пользователя по короткой ссылке на исходный URL (поддержка HTTP/HTTPS).
3	Дать владельцу возможность управлять своими ссылками: просмотр списка, поиск/фильтрация, отключение/включение и удаление.
4	Поддерживать срок действия ссылки (дата/время окончания) и автоматическое прекращение редиректа после истечения срока.
5	Поддерживать пользовательский алиас (кастомный суффикс) для короткой ссылки при условии уникальности.
6	Предоставить базовую аналитику по каждой ссылке: общее число переходов и простая динамика по времени (например, по дням).
7	Обеспечить прозрачную обработку ошибок: понятная страница «ссылка не найдена/отключена/истекла» и отсутствие «немых» ошибок.

Таблица 2. Функциональные требования

№	Функциональные требования / User Stories
1	• Как пользователь, я хочу открыть веб-страницу сервиса, ввести исходный URL и получить короткую ссылку, чтобы удобно делиться ею. • Как пользователь, я хочу сразу после создания видеть короткую ссылку и скопировать её одним действием, чтобы быстро использовать её вне сервиса.
2	• Как пользователь, я хочу перейти по короткой ссылке и быть перенаправленным на исходный URL, чтобы быстро открыть нужную страницу.
3	• Как владелец ссылки, я хочу видеть список созданных мной ссылок, чтобы управлять ими. • Как владелец ссылки, я хочу искать/фильтровать ссылки (например, по части URL или по статусу), чтобы быстро находить нужные. • Как владелец ссылки, я хочу отключать и включать редирект по ссылке, чтобы временно прекращать доступ. • Как владелец ссылки, я хочу удалять ссылку, чтобы она больше не была доступна.
4	• Как владелец ссылки, я хочу задавать срок действия (дату/время окончания) при создании, чтобы ссылка автоматически перестала работать после указанного момента. • Как система, я хочу прекращать редирект по истекшим ссылкам, чтобы пользователи не переходили по неактуальным адресам. • Как владелец ссылки, я хочу видеть срок действия в списке моих ссылок, чтобы понимать актуальность.
5	• Как владелец ссылки, я хочу указать пользовательский алиас (кастомный суффикс) при создании, чтобы ссылка была запоминаемой. • Как владелец ссылки, я хочу получить подсказку о допустимом формате алиаса, чтобы вводить корректные значения.
6	• Как владелец ссылки, я хочу видеть общее число переходов по каждой ссылке, чтобы оценивать интерес к ней. • Как владелец ссылки, я хочу видеть простую динамику переходов по дням, чтобы понимать тренд.
7	• Как пользователь, я хочу видеть понятную страницу ошибки, если ссылка не найдена, чтобы понимать причину. • Как пользователь, я хочу видеть понятную страницу ошибки, если ссылка отключена или истекла, чтобы понимать, почему редирект не выполняется.

Нефункциональные требования

Таблица 3. Нефункциональные требования

№	Группа требований	Нефункциональные показатели
1	Предоставить пользователям возможность преобразовывать длинные URL в короткие ссылки для удобного распространения в мессенджерах, социальных сетях и учебных/рабочих материалах.	• Совместимость интерфейса: страница создания короткой ссылки должна корректно работать в Chrome/Firefox/Edge (последние 2 версии) и на мобильных экранах. • Производительность создания: p95 время создания короткой ссылки ≤ 500 мс при нагрузке до 20 RPS. • Валидация ввода: сервис должен отклонять пустые и некорректные URL и показывать понятное сообщение об ошибке не позднее 1 секунды после отправки формы. • Надёжность хранения данных: соответствие «короткая ссылка $\rightarrow$ исходный URL» должно храниться в устойчивом хранилище и не теряться при перезапуске сервиса. • Уникальность результата: каждая созданная короткая ссылка должна быть уникальной в пределах сервиса.
2	Обеспечить быстрый и корректный редирект пользователя по короткой ссылке на исходный URL (поддержка HTTP/HTTPS).	• Производительность редиректа: p95 время обработки запроса редиректа ≤ 150 мс при нагрузке до 200 RPS (без учёта задержек внешней сети). • Надёжность: процент успешных редиректов (2xx/3xx) $\geq 99.5\%$ в месяц для доступных ссылок. • Корректность: редирект должен вести строго на сохранённый исходный URL без модификации (кроме нормализации пробелов/кодирования при хранении).

№	Группа требований	Нефункциональные показатели
3	Дать владельцу возможность управлять своими ссылками: просмотр списка, поиск/фильтрация, отключение/включение и удаление.	• Контроль доступа: пользователь должен иметь доступ только к своим ссылкам (просмотр/изменение/удаление). • Производительность списка: отображение списка ссылок (до 1000 ссылок на пользователя) должно работать с $p95 \leq 700$ мс. • Согласованность статуса: после отключения/удаления ссылка должна перестать редиректить не позже чем через 60 секунд.
4	Поддерживать срок действия ссылки (дата/время окончания) и автоматическое прекращение редиректа после истечения срока.	• Точность истечения: ссылка должна считаться истёкшей не позже чем через 60 секунд после заданного времени окончания. • Однозначность времени: система должна хранить и сравнивать время истечения в едином формате (например, UTC), а в UI явно показывать часовой пояс. • Поведение после истечения: при обращении к истекшей ссылке должна отображаться понятная страница ошибки (см. бизнес-требование про ошибки).
5	Поддерживать пользовательский алиас (кастомный суффикс) для короткой ссылки при условии уникальности.	• Ограничения алиаса: алиас должен состоять из символов [A-Z a-z 0-9 _ -], длина 3–32 символа. • Уникальность: система должна гарантировать уникальность алиаса в пределах домена сервиса (100%). • Производительность проверки: проверка/создание алиаса не должна увеличивать $p95$ время создания ссылки выше 600 мс.
6	Предоставить базовую аналитику по каждой ссылке: общее число переходов и простая динамика по времени (например, по дням).	• Актуальность статистики: счётчик переходов и агрегаты «по дням» должны обновляться не позже чем через 60 секунд после перехода. • Нагрузка: сбор статистики не должен увеличивать $p95$ время редиректа более чем на 20 мс относительно редиректа без статистики. • Хранение: статистика по дням должна храниться минимум 90 дней.
7	Обеспечить прозрачную обработку ошибок: понятная страница «ссылка не найдена/отключена/истекла» и отсутствие «немых» ошибок.	• Понятность: страницы ошибок должны содержать человекочитаемую причину («не найдена», «отключена», «истекла») без технических деталей. • Безопасность: страницы ошибок не должны раскрывать внутренние сведения (stack trace, SQL, конфигурацию). • Наблюдаемость: при ошибке должен формироваться лог-запись с временем, кодом ошибки и идентификатором короткой ссылки (если есть).

Ключевые количественные НФТ: создание короткой ссылки $p95 \leq 500$ мс при 20 RPS; создание с alias $p95 \leq 600$ мс; редирект $p95 \leq 150$ мс при 200 RPS и успешность $\geq 99,5\%$; список до 1000 ссылок $p95 \leq 700$ мс; обновление статуса и статистики не позднее 60 секунд; хранение статистики не менее 90 дней.

Расчёт ресурсов

Расчёт ресурсов выполнен по значениям из Excel-таблицы. В сравнении с ранними пояснительными материалами приоритет имеют значения таблицы: MAU = 100000000, DAU = 10000000, активное окно пиковой нагрузки - 16 часов в сутки. Из-за этого расчётная нагрузка выше первичных НФТ-лимитов 20 RPS для создания и 200 RPS для редиректа.

Таблица 4. Ключевые результаты расчёта ресурсов

№	Характеристика	Значение	Единица / вывод
1	MAU	100000000	активных пользователей в месяц
2	DAU	10000000	активных пользователей в день
3	Активные часы пиковой нагрузки	16	часов в сутки
4	Создание коротких ссылок всеми пользователями	1333333,33 в день; 833333,33 в час; 231,48 RPS	ключевая write-операция
5	Переходы по коротким ссылкам всеми пользователями	20000000 в день; 1250000 в час; 3 472,22 RPS	ключевая read-операция
6	Просмотр кабинета / списка ссылок всеми пользователями	20000000 в день; 1250000 в час; 347,22 RPS	управленческий read-сценарий
7	Средний размер запроса/ответа create	2 КБ / 1 КБ	клиентский трафик
8	Средний размер запроса/ответа redirect	0,5 КБ / 1 КБ	клиентский трафик
9	Средний размер запроса/ответа cabinet	1 КБ / 10 КБ	клиентский трафик
10	Ingress клиентского трафика	20370,37 Кбит/с $\approx$ 20,37 Мбит/с	сумма create + redirect + cabinet
11	Egress клиентского трафика	57407,41 Кбит/с $\approx$ 57,41 Мбит/с	основная доля - redirect и кабинет
12	Read/Write ratio	16,5 : 1	система read-heavy
13	CDN	не требуется; 0 CDN-серверов	MVP не геораспределён и не хранит BLOB-контент
14	Итоговый SSD для ключевых данных	4,93 TB / 14,46 TB / 24,00 TB	горизонт 1 / 3 / 5 лет
15	HDD для ключевых данных	0 MB	BLOB-данные не используются
16	PostgreSQL-ноды для SSD-данных	3 / 8 / 12	при полезной ёмкости 2 TB на ноду
17	Redis hot-set	5859,38 MB $\approx$ 5,86 GB	горячие route_key и временный click stream
18	Использование лимита НФТ по созданию	1157,4% от лимита 20 RPS	требуется пересмотр capacity или масштабирование
19	Использование лимита НФТ по редиректам	1736,1% от лимита 200 RPS	самый напряжённый сценарий

Архитектурный приоритет: для основного сценария редиректа выбран Availability; статистика допускает Eventual Consistency; уникальность public route_key и пользовательского alias обеспечивается Strong Consistency через уникальный индекс в PostgreSQL. При расчётной нагрузке предпочтителен предгенерированный пул route_key, чтобы не выполнять полную генерацию в синхронном create-path.

Бизнес- и прикладные домены

Таблица 5. Бизнес- и прикладные домены

№	Бизнес-домен	Функциональные домены	Прикладные домены
1	Создание и управление ссылками	• Создание короткой ссылки • Валидация и нормализация исходного URL • Генерация короткого кода / проверка пользовательского алиаса • Управление сроком действия ссылки • Просмотр списка ссылок и поиск • Изменение статуса ссылки и удаление • Получение и копирование результата сокращения	• Web UI формы создания и списка ссылок • API/endpoint создания и обновления ссылок • Генератор short-code и проверка уникальности алиаса • Таблица SHORT_LINKS в РСУБД • Пагинация, фильтрация и поиск по ссылкам • Фоновая задача деактивации истёкших ссылок • Сервис выдачи готовой короткой ссылки и метаданных
2	Переход по коротким ссылкам	• Разрешение короткого кода в исходный URL • Выполнение HTTP-редиректа • Обработка ссылок «не найдена / отключена / истекла» • Поддержка пользовательского алиаса в URL	• Redirect handler • Кэш горячих ссылок (Redis) • Страницы ошибок и корректные HTTP-коды • Индекс/поиск по short-code и alias
3	Личный кабинет и доступ	• Идентификация владельца ссылок • Разграничение доступа к своим ссылкам • Просмотр собственных ссылок, статистики	• Сессии / аутентификация пользователя • Middleware авторизации и проверки прав доступа • Таблицы USERS и связи user -> short link
4	Аналитика переходов	• Учёт переходов по ссылкам • Агрегация статистики по дням • Отображение статистики владельцу	• Счётчик переходов и запись события клика • Таблица LINK_DAILY_STATS • API/страница просмотра статистики

Выбор технологий и проектирование

High-level design

На абстрактном уровне система разделена на три контура: управление ссылками, быстрый редирект и фоновая аналитика. Контур редиректа изолирован от кабинета, так как это самый чувствительный к latency сценарий. Аналитика вынесена из синхронного redirect-path, чтобы сбор кликов не увеличивал время ответа.



Рисунок 1. Abstract High-level design URL Shortener

На concrete HLD выделены контейнеры Reverse proxy / edge, Management app, Redirect app, Background worker, PostgreSQL и Redis. PostgreSQL является источником истины; Redis используется как ускоряющий слой и как временный поток событий, но не как критическое хранилище данных.

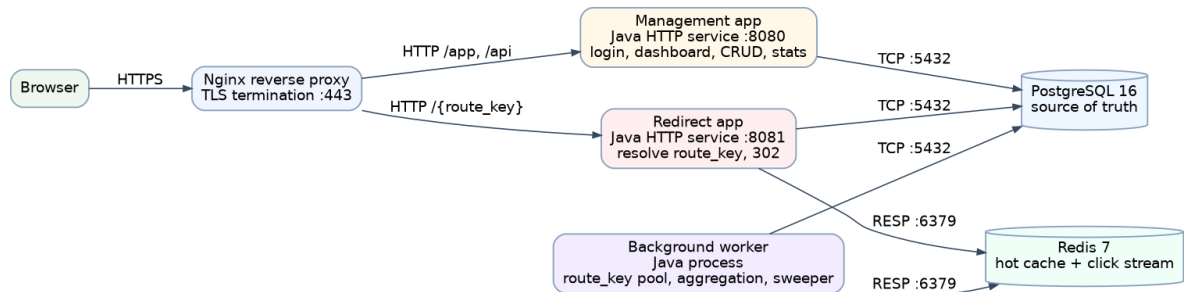


Рисунок 2. Concrete High-level design / container-level C4

Low-level design

Компонентный уровень отражает финальный состав запускаемых компонентов и middleware. На связях указаны протоколы и порты, соответствующие требованиям лабораторной работы.

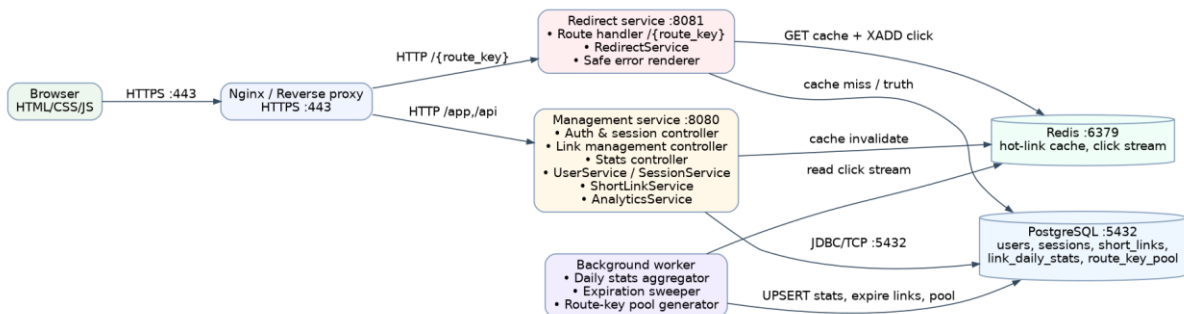


Рисунок 3. Low-level design / component-level C4

Таблица 6. Протоколы и порты low-level design

Связь	Протокол / порт	Назначение
Browser → Reverse proxy	HTTPS :443	Вход в систему, личный кабинет и публичные переходы
Reverse proxy → Management service	HTTP :8080	Маршруты /app и /api/*
Reverse proxy → Redirect service	HTTP :8081	Маршрут /{route_key} для lookup и 302
Management service → PostgreSQL	PostgreSQL/TCP :5432	Users, sessions, short_links и чтение статистики
Management service → Redis	RESP/TCP :6379	Инвалидация hot cache при изменении статуса ссылки
Redirect service → Redis	RESP/TCP :6379	Чтение hot cache и публикация click event
Redirect service → PostgreSQL	PostgreSQL/TCP :5432	Fallback на cache miss и источник истины
Background worker → Redis	RESP/TCP :6379	Чтение click stream
Background worker → PostgreSQL	PostgreSQL/TCP :5432	UPSERT link_daily_stats, истечение ссылок, пополнение пула ключей

Описание используемых технологий

Таблица 7. Используемые технологии

Технология	Где используется	Обоснование
Java SE / com.sun.net.httpserver	Management service, Redirect service, Edge proxy, Background worker	Позволяет реализовать учебный прототип без тяжёлых framework-зависимостей; код разделён на web/service/store/model слои.
PostgreSQL 16	Источник истины	Нужны транзакции, уникальные ограничения для route_key/alias, внешние ключи, индексы и UPSERT статистики.
Redis 7	Hot-link cache и click stream	Снижает количество lookup в PostgreSQL и позволяет асинхронно фиксировать клики без удлинения redirect-path.
Nginx 1.27	Reverse proxy / edge	TLS termination, маршрутизация /app в Management app и /{route_key} в Redirect app; основа для upstream scaling.
Docker Compose	Развёртывание распределённого стенда	Запускает PostgreSQL, Redis, Management service, Redirect service, Background worker и reverse proxy.
HTML/CSS/JavaScript	Клиентский интерфейс	Формы создания, кабинет, фильтрация, копирование ссылки, страницы ошибок и статистика.
Собственный load-test runner	Нагрузочные проверки	Генерирует HTTPS-запросы к стенду, создаёт тестовые данные, считает RPS, p50/p90/p95/p99, успешность и покрытие требований.

Разработка распределённой системы

Прототип реализован как учебная распределённая система. В демонстрационном режиме все компоненты могут запускаться в одном JVM с файловым snapshot-хранилищем; в целевом распределённом режиме используются отдельные процессы/контейнеры Management service, Redirect service, Background worker, PostgreSQL, Redis и Nginx reverse proxy.

Основные исходные файлы: urlshortener/web/ManagementServer.java, RedirectServer.java, EdgeProxyServer.java; service-слой ShortLinkService, RedirectService, AnalyticsService, BackgroundWorker; store-слой PostgresDataStore; deploy/docker-compose-full.yml; deploy/initdb/01-schema.sql; deploy/nginx/nginx-full.conf.

Итоговый объём реализации после архитектурной проработки не сокращён: в MVP оставлены создание короткой ссылки, редирект, кабинет владельца, пользовательский alias, срок действия, базовая аналитика и безопасная обработка ошибок. Вне scope: публичный token API, anti-abuse/anti-spam подсистемы, биллинг, multi-tenant custom domains и модерация контента.

Описание схемы данных

Концептуальная модель включает сущности USER, USER_SESSION, SHORT_LINK и LINK_DAILY_STAT. Это покрывает кабинет владельца, сессии, связь user → short_link, срок действия, alias и базовую аналитику по дням.

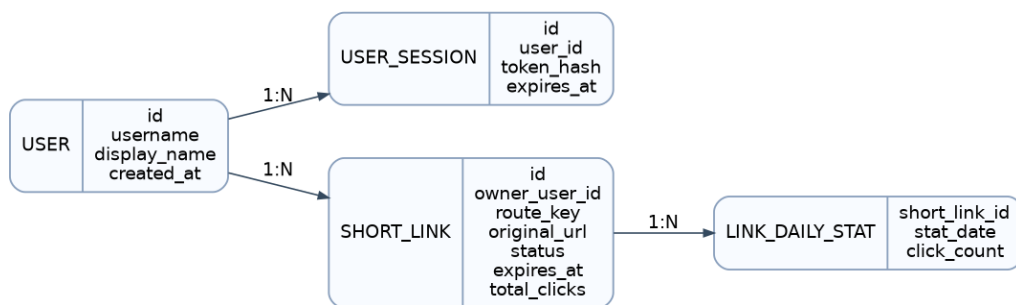


Рисунок 4. Концептуальная ER-модель

Физическая модель уточняет типы данных, первичные и внешние ключи, уникальные ограничения и индексы. Ключевое решение - единое поле `route_key` с уникальным ограничением: оно обслуживает как автоматический код, так и пользовательский alias. Для `link_daily_stats` выбран составной PK (`short_link_id`, `stat_date`), что упрощает UPSERT дневной статистики.

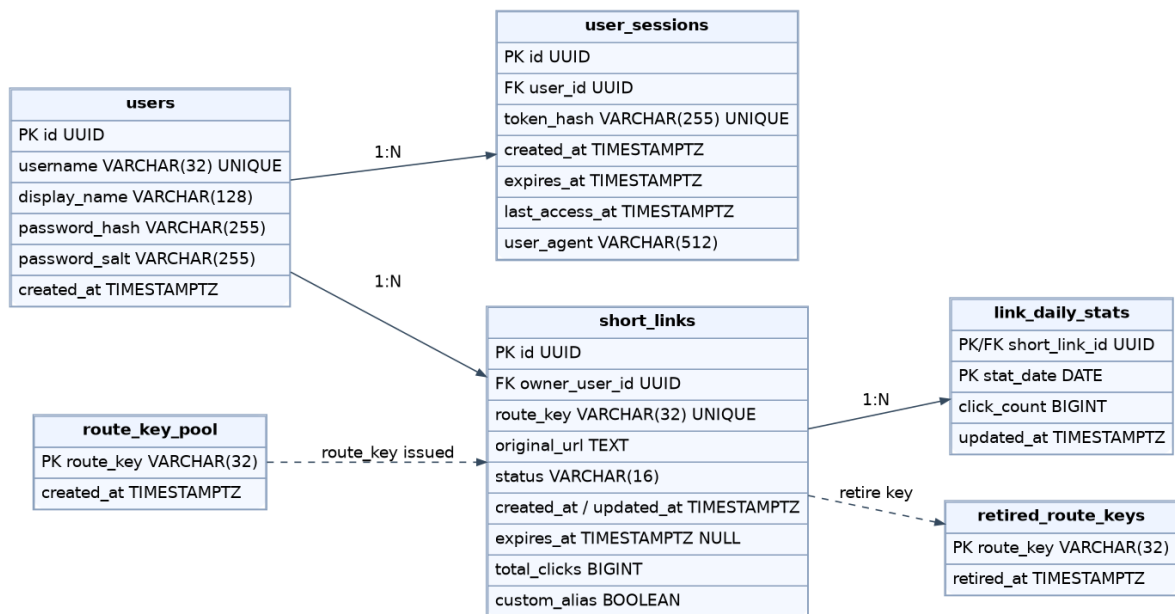


Рисунок 5. Физическая ER-модель

Таблица 8. Описание физических таблиц

Таблица	Назначение	Ключевые поля и индексы
users	Зарегистрированные владельцы ссылок	PK id; username UNIQUE; display_name; password hash/salt; created at
user_sessions	Сессии личного кабинета	PK id; FK user_id; token_hash UNIQUE; expires_at; last_access at; user_agent
short_links	Основная таблица коротких ссылок	PK id; FK owner_user_id; route_key UNIQUE; original_url; status; expires_at; total_clicks; custom_alias; индексы owner_user id+created at, owner user id+status+created at, status+expires at
link_daily_stats	Агрегаты переходов по дням	PK (short_link_id, stat_date); FK short_link_id; click count; updated at
route_key_pool	Предгенерированный пул свободных route_key	PK route_key; created_at
retired_route_keys	История выведенных из обращения ключей	PK route_key; retired_at

Описание контрактов серверного API

Swagger-спецификация для прототипа не выделялась: интерфейс реализован как form-based Web UI с серверными HTML-страницами и публичным redirect endpoint. Ниже приведены фактические контракты HTTP-маршрутов.

Таблица 9. Контракты серверного HTTP API

Метод	Маршрут	Параметры	Ответ / эффект
GET	/health	-	200 OK, текстовый health-check
GET	/	-	302 на /app/login или /app/dashboard
GET/POST	/app/register	username, displayName, password, confirmPassword	HTML регистрация; создание пользователя и сессии
GET/POST	/app/login	username, password	HTML логин; создание session cookie
POST	/app/logout	session cookie	Удаление сессии, очистка cookie
GET	/app/dashboard	q, status, page, size	Кабинет владельца, создание ссылки, список, поиск и фильтрация
POST	/app/links/create	originalUrl, alias, expiresAtMoscow	Создание short link, выдача short URL, 400 при валидации
POST	/app/links/toggle	linkId, targetState	Включение/отключение ссылки владельца
POST	/app/links/delete	linkId	Удаление ссылки владельца
POST	/app/links/history-delete	selectedLinkId[], q, status, page, size	Массовое удаление записей истории
GET	/app/links/stats	linkId	Страница статистики ссылки за последние 90 дней
GET	/app/lang	value, redirect	Выбор языка интерфейса и возврат на исходную страницу
GET	/ {route_key}	route_key в path	302 Location на original_url либо безопасная HTML-страница ошибки 404/410/423

Интерфейс взаимодействия с конечным пользователем

Screen-flow клиентской части покрывает регистрацию/логин, кабинет владельца, создание короткой ссылки, список и фильтрацию, просмотр статистики и безопасные страницы ошибок.

Рисунок 6. Экран входа пользователя

Сократитель ссылок

Учебный MVP по проектированию распределённых систем

EnRu

Учебный MVP по проектированию распределённых систем

Регистрация

Создайте аккаунт владельца для управления короткими ссылками.

Имя пользователя

Отображаемое имя

Пароль

Подтверждение пароля

Создать аккаунт

Уже есть аккаунт? [Перейти ко входу](#)

Рисунок 7. Экран регистрации пользователя

Сократитель ссылок

Учебный MVP по проектированию распределённых систем

En

Py

Текущий пользователь lin

lin

MVP-покрытие: создание короткой ссылки, 302-редирект, alias, срок действия, кабинет владельца, базовая аналитика и безопасные страницы ошибок.

Выйти

Вход выполнен.

Создать короткую ссылку

Исходный URL

https://example.com/very/long/link

Пользовательский alias (необязательно)

my-course-link

Допустимый формат: [A-Za-z0-9_-], длина 3–32. Оставьте пустым, чтобы использовать предгенерированный route_key.

Время истечения (по московскому времени, необязательно)

2026-03-18T15:30:00

Пример: 2026-03-18T15:30:00 или 2026-03-18T15:30:00+03:00. И интерфейс, и сервер трактуют значение по московскому времени.

Создать ссылку

Фильтрация и поиск

Поиск по route_key или URL

alias или часть исходного URL

Статус

Все

Применить фильтры

Мои ссылки

Страница 1 из 1 • всего 0

Удалить историю

КОРОТКАЯ ССЫЛКА	ИСХОДНЫЙ URL	СТАТУС	ВСЕГО ПЕРЕХОДОВ	ПОЛЬЗОВАТЕЛЬСКИЙ ALIAS	ВРЕМЯ ИСТЕЧЕНИЯ	СОЗДАНО	ДЕЙСТВИЯ
По текущему фильтру ссылки не найдены.							

Рисунок 8. Кабинет владельца: создание ссылки, фильтрация и список

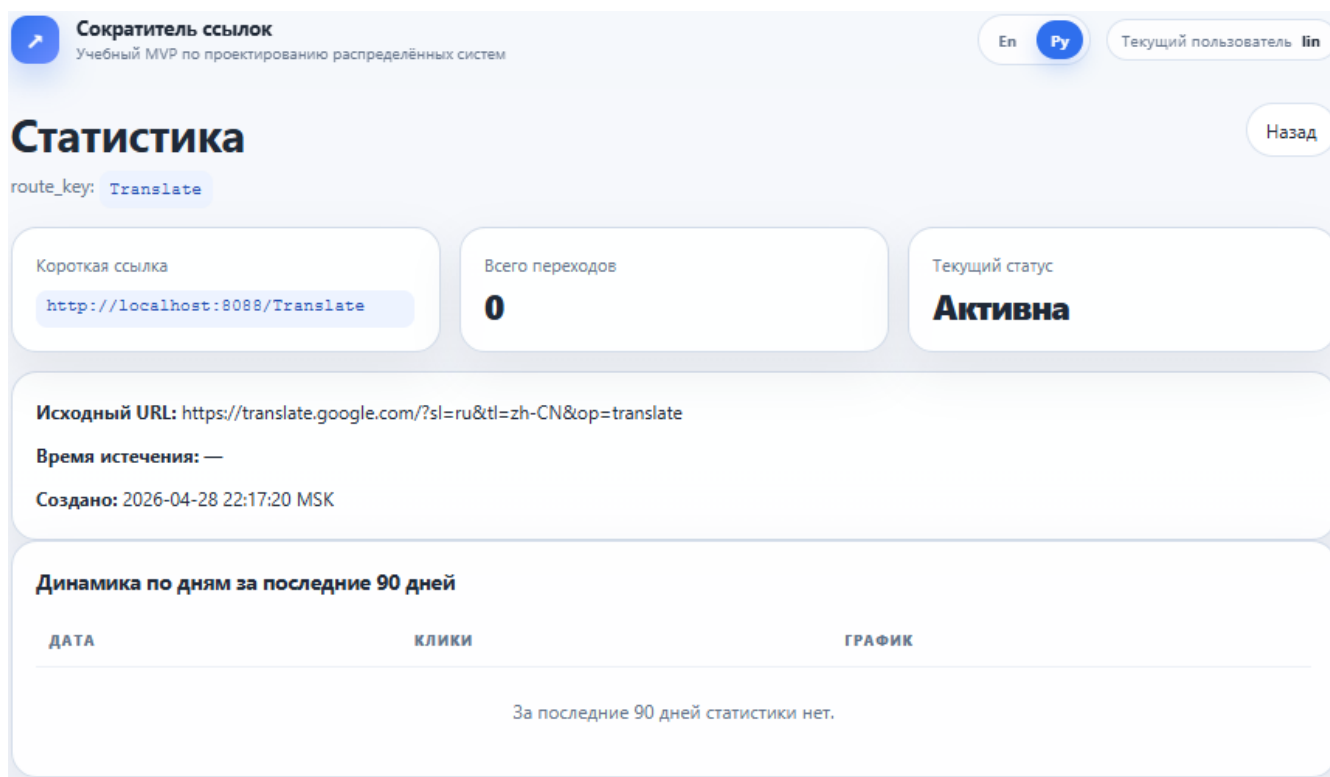


Рисунок 9. Страница статистики короткой ссылки

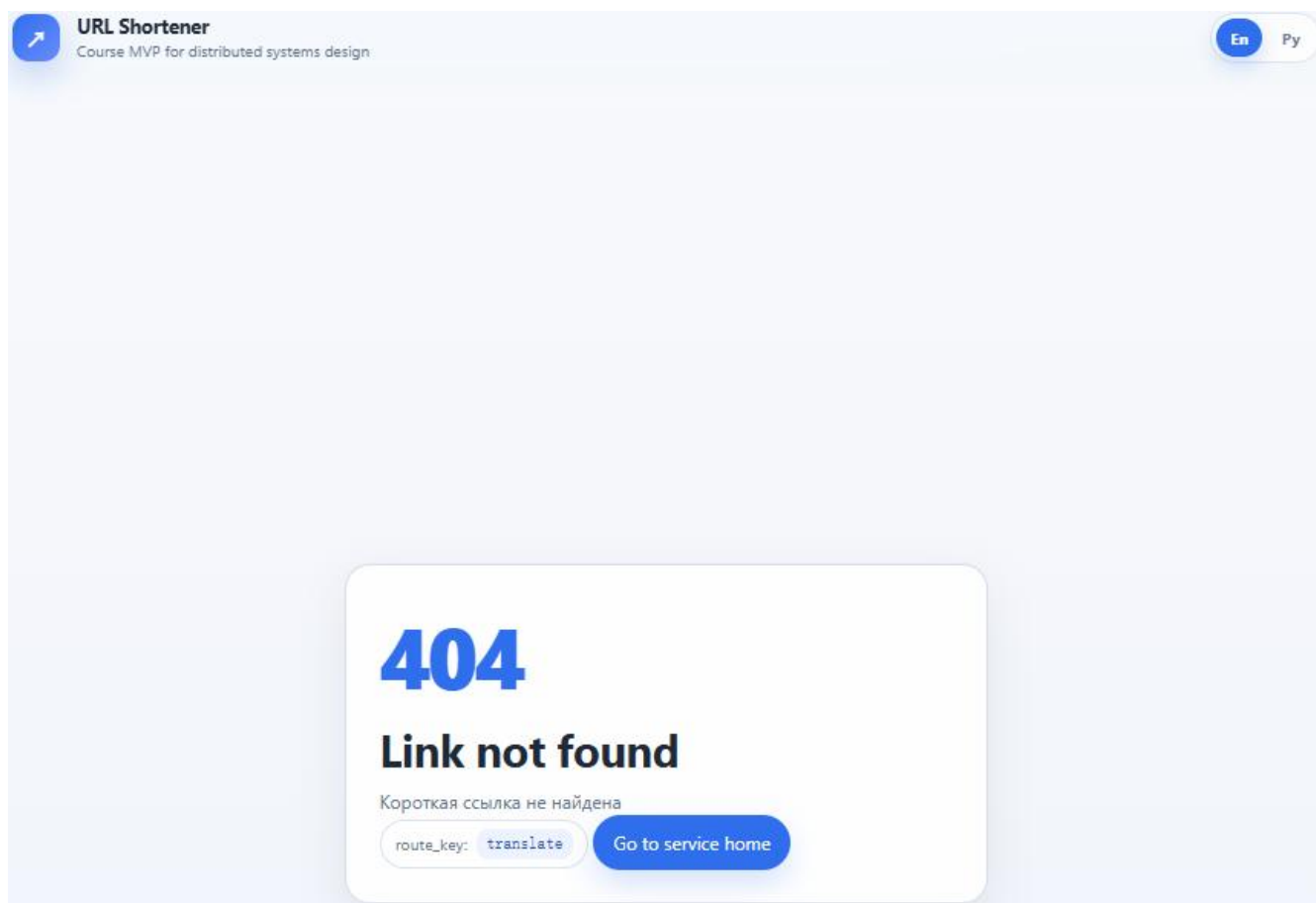


Рисунок 10. Безопасная публичная страница ошибки для короткой ссылки

Таблица 10. Screen-flow и покрытие функциональных требований

Шаг	Экран / действие	Покрываемые требования
1	Регистрация или вход в личный кабинет	Личный кабинет, идентификация владельца, контроль доступа
2	Создание короткой ссылки с optional alias и expiresAt	BT1, BT4, BT5, валидация URL и alias
3	Копирование готовой короткой ссылки	Удобное распространение ссылки
4	Переход по <code>{route key}</code>	Быстрый 302-редирект или безопасная ошибка
5	Список, поиск, фильтрация, включение/отключение/удаление	Управление ссылками владельца и согласованность статуса
6	Просмотр статистики	Общее число переходов и динамика по дням

Тестирование

Архитектурное соответствие нефункциональным требованиям

Архитектурные решения проектировались под заявленные НФТ, но расчёт ресурсов по Excel показывает, что целевые пиковые RPS для выбранного масштаба существенно превышают первичные лимиты НФТ. Поэтому ниже разделяются: проектные механизмы удовлетворения требований и фактическое состояние учебного стенда по результатам нагрузочных испытаний.

Таблица 11. Архитектурные механизмы удовлетворения НФТ

№	Требование	Шаблон / свойство архитектуры	Вывод
1	Создание ссылки $p95 \leq 500$ мс при 20 RPS; alias $p95 \leq 600$ мс	Короткий write-path, UQ-индекс route_key, предгенерированный пул route_key, отдельный Management app	Механизм корректен; расчётные 231 RPS требуют масштабирования и настройки пулов.
2	Редирект $p95 \leq 150$ мс при 200 RPS, успешность $\geq 99,5\%$	Отдельный Redirect app, Redis hot cache, fallback в PostgreSQL, отсутствие синхронной аналитики	Проектно достижимо; текущий стенд показал хвостовые таймауты и недобор RPS.
3	Список до 1000 ссылок $p95 \leq 700$ мс	Индексы owner_user_id+created_at и owner_user_id+status+created_at; пагинация	Индексированная выборка быстрая; нужны устойчивые пулы и back-pressure.
4	Уникальность alias 100%	Единое поле route_key с UNIQUE в PostgreSQL	Strong Consistency на точке записи.
5	Истечение ссылки ≤ 60 секунд	Проверка expires at при каждом redirect + background sweeper	После истечения redirect не выполняется.
6	Обновление статистики ≤ 60 секунд и отсутствие влияния на redirect > 20 мс	Redis stream + Background worker + батчевый UPSERT	Eventual Consistency без удлинения redirect-path.
7	Безопасные страницы ошибок	Error renderer без stack trace/SQL; структурированное логирование	Закрывает понятность, безопасность и наблюдаемость.

Нагрузочные испытания

Нагрузочное тестирование выполнялось через reverse проху распределённого стенда с базовым URL <https://localhost>. Покрыты 27 из 27 обязательных проверок требований. Из-за ограничений учебного стенда длительность контрольных сценариев составила 60 секунд, а sarasity-сценариев - 30 секунд; это меньше рекомендуемых 15 минут, поэтому результаты используются как диагностика прототипа и инфраструктуры, а не как промышленная сертификация НФТ.

- Контрольный запуск: 8 сценариев, 1 пройден и 7 не пройдены.
- Sarasity-запуск: 11 сценариев, 2 пройдены и 9 не пройдены.
- Основные причины непрохождения: недостаточная успешность, недобор фактической RPS и хвостовые таймауты около 10 секунд.

Таблица 12. Результаты контрольного нагрузочного запуска

Сценарий	Цель RPS	Факт RPS	Успешность	p95, мс	Лимит, мс	Результат
Создание короткой ссылки без alias	20	17,23	97,50%	24	500	НЕ ПРОЙДЕНО
Создание короткой ссылки с alias	20	18,00	93,75%	10000	600	НЕ ПРОЙДЕНО
Редирект по горячим коротким ссылкам	200	146,85	55,15%	10000	150	НЕ ПРОЙДЕНО
Список ссылок до 1000 записей	50	43,49	95,90%	90	700	НЕ ПРОЙДЕНО
Страницы ошибок короткой ссылки	20	20,01	98,92%	13	1000	НЕ ПРОЙДЕНО
Валидация некорректного URL	20	20,01	100,00%	28	1000	ПРОЙДЕНО
Просмотр статистики ссылки	20	17,27	95,08%	5128	700	НЕ ПРОЙДЕНО
Отключение, включение и удаление	10	10,01	97,33%	22	700	НЕ ПРОЙДЕНО

В контрольном запуске полностью пройден сценарий валидации некорректного URL. Часть сценариев имела низкий p95, но не была засчитана из-за успешности ниже целевого уровня или недобора фактической RPS.

p95 задержка по сценариям

Полоса = фактическое значение; вертикальная линия = целевое значение. Единица: миллисекунды

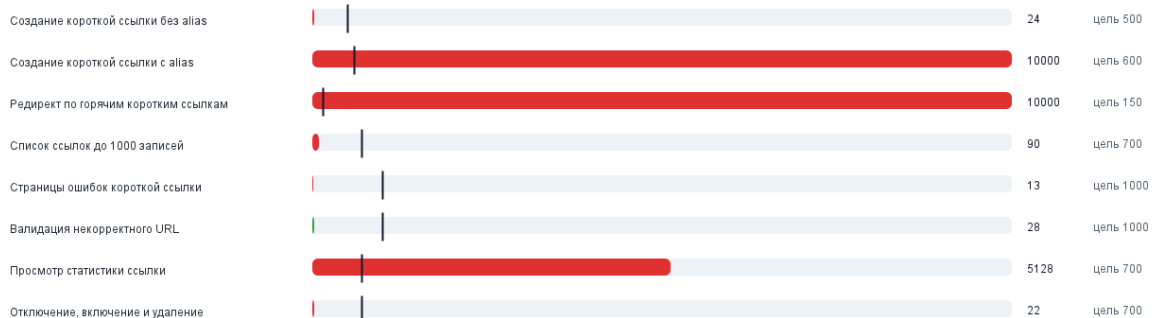


Рисунок 11. Контрольный запуск: p95 задержка по сценариям

фактическая RPS по сценариям

Полоса = фактическое значение; вертикальная линия = целевое значение. Единица: запросов в секунду

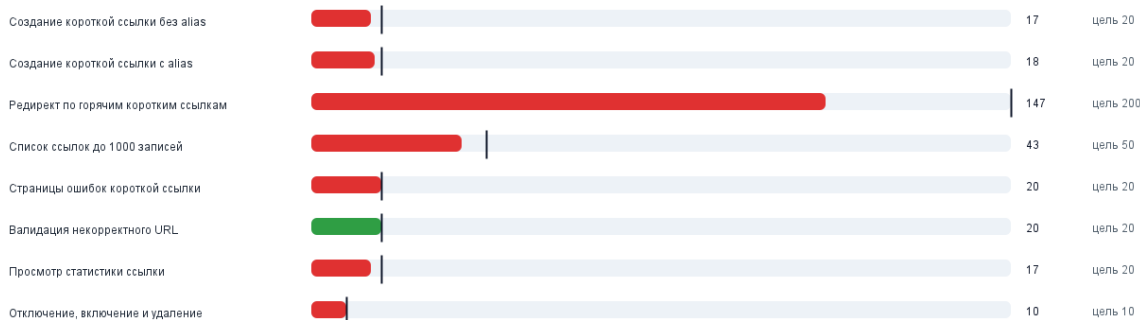


Рисунок 12. Контрольный запуск: фактическая RPS по сценариям

успешность по сценариям

Полоса = фактическое значение; вертикальная линия = целевое значение. Единица: проценты



Рисунок 13. Контрольный запуск: успешность по сценариям

Стресс-/saracity-тестирование

Отдельный saracity-запуск выполнен на том же стенде и включает расчётные значения из Excel: создание 231 RPS, редирект 3472 RPS и кабинет 347 RPS. Он показывает не только соответствие первичным НФТ, но и фактический запас прототипа на уровне расчётной ресурсной модели.

Таблица 13. Результаты saracity-запуска

Сценарий	Цель RPS	Факт RPS	Успешность	p95, мс	Лимит, мс	Результат
Создание короткой ссылки без alias	20	15,05	97,83%	23	500	НЕ ПРОЙДЕНО
Создание короткой ссылки с alias	20	17,45	96,67%	5132	600	НЕ ПРОЙДЕНО
Редирект по горячим коротким ссылкам	200	100,55	84,33%	10004	150	НЕ ПРОЙДЕНО
Список ссылок до 1000 записей	50	50,01	99,27%	17	700	ПРОЙДЕНО
Страницы ошибок короткой ссылки	20	15,43	97,50%	16	1000	НЕ ПРОЙДЕНО
Валидация некорректного URL	20	20,02	98,67%	26	1000	НЕ ПРОЙДЕНО
Просмотр статистики ссылки	20	16,93	96,00%	17	700	НЕ ПРОЙДЕНО
Отключение, включение и удаление	10	9,95	100,00%	25	700	ПРОЙДЕНО

Сценарий	Цель RPS	Факт RPS	Успешность	p95, мс	Лимит, мс	Результат
Расчёт ресурсов: создание 231 RPS	231	114,83	91,49%	10003	500	НЕ ПРОЙДЕНО
Расчёт ресурсов: редирект 3472 RPS	3472	910,59	10,01%	656	150	НЕ ПРОЙДЕНО
Расчёт ресурсов: кабинет 347 RPS	347	208,23	34,27%	561	700	НЕ ПРОЙДЕНО

Дополнительные capacity-сценарии не достигли целевых значений: создание показало 114,83 RPS из 231 RPS, редирект - 910,59 RPS из 3472 RPS, кабинет - 208,23 RPS из 347 RPS. По успешности наиболее проблемны redirect-capacity и cabinet-capacity.

p95 задержка по сценариям

Полоса = фактическое значение; вертикальная линия = целевое значение. Единица: миллисекунды

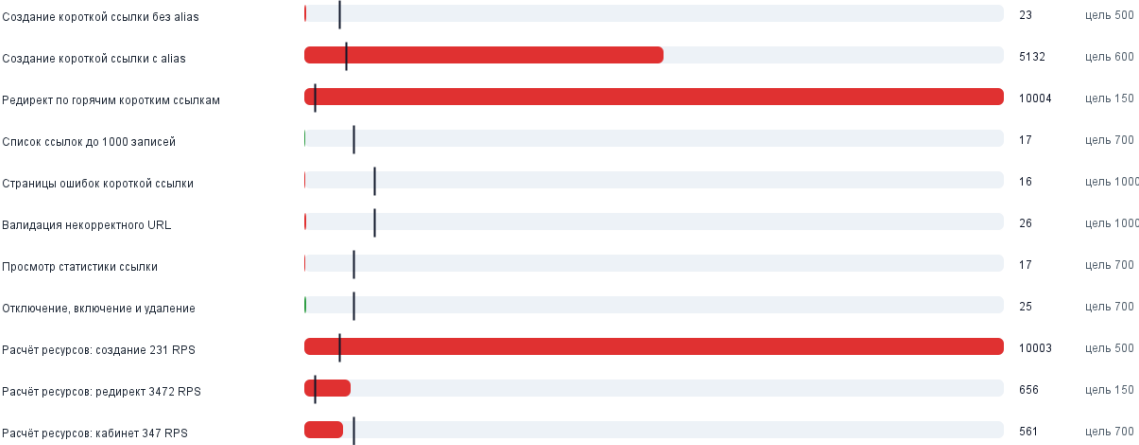


Рисунок 14. Capacity-запуск: p95 задержка по сценариям

фактическая RPS по сценариям

Полоса = фактическое значение; вертикальная линия = целевое значение. Единица: запросов в секунду

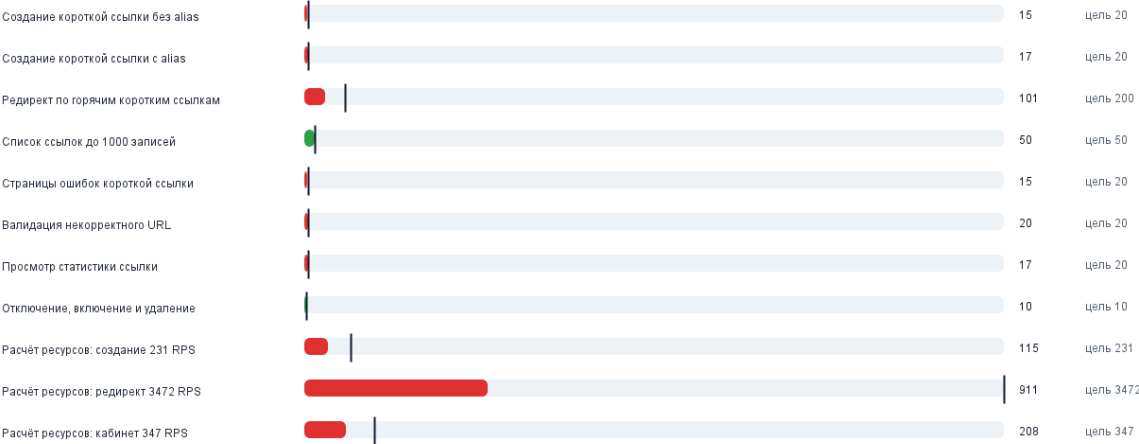


Рисунок 15. Capacity-запуск: фактическая RPS по сценариям

успешность по сценариям

Полоса = фактическое значение; вертикальная линия = целевое значение. Единица: проценты



Рисунок 16. Saracity-запуск: успешность по сценариям

Нагрузка x10 от плановой

Если считать x10 от первичных НФТ-лимитов, то целевые значения составляют примерно 200 RPS для создания, 2000 RPS для редиректа и 500 RPS для списка. Расчёт ресурсов уже требует 231,48 RPS для создания и 3472,22 RPS для редиректа, то есть по двум критичным сценариям ресурсная модель фактически равна или выше x10 от первичного плана. Текущий стенд x10-нагрузку не выдерживает без доработок: saracity-запуск показал 114,83 RPS для создания, 910,59 RPS для редиректа и 208,23 RPS для кабинета.

Для обработки x10 без существенной потери производительности необходимо: запустить несколько экземпляров Redirect app и Management app за Nginx upstream; включить keepalive и health-check; заменить учебный PostgreSQL wire-client на JDBC + HikariCP или PgBouncer; увеличить и разнести пулы соединений по сервисам; прогреть Redis hot cache; использовать pipeline для click events; масштабировать PostgreSQL по партициям/шардам и read-replicas; внедрить back-pressure, таймауты и rate limiting на edge.

Эксплуатация

Расчёт доступности

Доступность рассчитана по последовательным цепочкам взаимодействия. Redis не включён в критический контур редиректа: при его недоступности сервис продолжает работать через PostgreSQL с деградацией latency. Для аналитики Redis и Background worker входят в цепочку, поэтому доступность аналитического контура ниже.

Таблица 14. Расчёт доступности ключевых операций

№	Ключевая операция	Доступность, %	Перечень сервисов	Формула расчёта	Действия для повышения доступности
1	Создание и управление ссылкой	99,8001%	Reverse proxy → Management app → PostgreSQL	$0,999 \times 0,9995 \times 0,9995 \times 100$	2+ edge/proxy, несколько Management app, PostgreSQL HA, PgBouncer, health-check.
2	Редирект по короткой ссылке	99,8001%	Reverse proxy → Redirect app → PostgreSQL	$0,999 \times 0,9995 \times 0,9995 \times 100$	Несколько Redirect app за upstream, Redis cache как деградационный ускоритель, PostgreSQL HA.
3	Актуальная аналитика / обновление статистики	99,2018%	Reverse proxy → Management app → PostgreSQL → Redis → Background worker	$0,999 \times 0,9995 \times 0,9995 \times 0,999 \times 0,995 \times 100$	Redis HA, worker group, идемпотентный UPSERT, повторная обработка событий.
4	Критический пользовательский контур	99,8001%	Минимум из create/manage и redirect	$\text{MIN}(99,8001\%; 99,8001\%)$	Повысить избыточность приложений и СУБД; убрать единые точки отказа.

Таблица 15. Входные значения доступности компонентов

Компонент	Допущение доступности
Reverse proxy / edge	99,90%
Management app	99,95%
Redirect app	99,95%
PostgreSQL	99,95%
Redis cache / stream	99,90%
Background worker	99,50%

Анализ узких мест

Ниже приведены основные узкие места и решения: фиксируется проблема, решение и ожидаемое последствие для архитектуры.

Таблица 16. Узкие места и work-around

№	Описание проблемы	Предлагаемое решение
1	Расчётная нагрузка из Excel превышает первичные НФТ-лимиты: 231 RPS create и 3472 RPS redirect.	Пересмотреть capacity-targets, масштабировать Management/Redirect горизонтально, использовать upstream LB, back-pressure и повторное тестирование на целевых RPS.
2	Redirect hot path показывает хвостовые таймауты и низкую успешность при нагрузке.	Несколько Redirect app, Redis pre-warm, keepalive, короткие таймауты, отдельный executor pool, контроль очередей и rate limiting.
3	Учебные пулы соединений и HTTP-пулы ограничивают пропускную способность стенда.	JDBC + HikariCP или PgBouncer; отдельные лимиты пулов для Management, Redirect и Worker; настройка executor pool.
4	DAT_SHORT_LINKS и общий SSD достигают около 24 TB за 5 лет; требуется 12 PostgreSQL-нод полезной ёмкостью 2 TB.	Партиционирование по времени и/или sharding по route_key/hash; архивирование неактивных ссылок; отдельные индексы для owner/status и route key.
5	Уникальность route_key/alias при высокой конкуренции может создавать конфликтные записи.	Единый UNIQUE(route_key), предгенерированный пул route_key, retry при конфликте, retired_route_keys для недопущения повторного использования.
6	Сбор статистики может повлиять на latency редиректа.	Оставить click event асинхронным через Redis stream, батчевый UPSERT link_daily_stats, ограничение длины stream, отдельный worker group.
7	Отключение/удаление/истечение ссылки требует согласованности с Redis cache.	Инвализация cache при изменении статуса, короткий TTL hot cache, проверка expires_at в redirect, sweeper для просроченных ссылок.
8	Ошибки публичных ссылок не должны раскрывать внутренние данные.	Единый Safe error renderer, запрет stack trace/SQL в HTML, структурированное логирование с route key и кодом ошибки.

1. Изоляция redirect-path. Контекст: редирект является критическим сценарием с $p95 \leq 150$ мс и расчётной нагрузкой 3472 RPS. Решение: выделить Redirect service, использовать Redis hot cache и асинхронную фиксацию click event. Последствие: Redirect app можно масштабировать независимо от кабинета; Redis не становится источником истины.
2. Strong consistency для route_key/alias. Контекст: alias должен быть уникальным на 100%. Решение: хранить alias и автоматический код в едином поле route_key с UNIQUE-ограничением PostgreSQL. Последствие: проверка уникальности централизована; при высокой нагрузке нужен предгенерированный пул и retry.
3. Асинхронная аналитика. Контекст: статистика должна обновляться в течение 60 секунд, но не должна увеличивать p95 редиректа более чем на 20 мс. Решение: писать click event в Redis stream и агрегировать worker-ом батчами в link_daily_stats. Последствие: статистика eventual consistent, а redirect-path остаётся коротким.

Выводы

В рамках курсовой работы спроектирован и реализован учебный прототип распределённой системы URL Shortener. Были собраны требования, выделены бизнес- и прикладные домены, построены abstract/concrete high-level design, low-level design, концептуальная и физическая модели данных, рассчитаны ресурсы, доступность и узкие места.

На уровне реализации прототип поддерживает создание коротких ссылок, редирект, личный кабинет владельца, срок действия, пользовательский alias, базовую аналитику и безопасные страницы ошибок. Распределённый стенд использует Nginx, Management app, Redirect app, Background worker, PostgreSQL и Redis.

Расчёт ресурсов по финальной Excel-таблице показывает MAU 100 млн и DAU 10 млн, а ключевые RPS - 231,48 для создания ссылок, 3472,22 для редиректа и 347,22 для кабинета. Эти значения значительно выше первичных НФТ-лимитов, поэтому для промышленного соответствия требуется горизонтальное масштабирование, донастройка пулов, Redis pipeline и PostgreSQL HA/sharding.

Нагрузочное тестирование покрыло 27 из 27 проверок требований, но текущая конфигурация стенда прошла только 1 из 8 контрольных сценариев и 2 из 11 capacity-сценариев. Основные проблемы - хвостовые таймауты, недобор фактической RPS и недостаточная успешность. Следовательно, функциональное покрытие и архитектурная декомпозиция реализованы, но эксплуатационная конфигурация прототипа должна быть доработана для полного выполнения НФТ под расчётной нагрузкой.